# Best Practices / Clean Code

Practical guidelines for code that is easier to understand, test and change. Each guideline includes a short example.

---

## 1. Choose meaningful names

A variable or function name should explain its purpose, so readers do not have to guess the intent from every line.

### Less clear

```
x = len(students)
y = x > 25
```

### Clearer

```
student_count = len(students)
is_large_group = student_count > 25
```

---

## 2. Give a function one clear responsibility

A function that reads a file, transforms data, prints output and writes a report is harder to test. Separate steps that have distinct responsibilities.

### Too much in one function

```
from pathlib import Path

def process_students(path: Path) -> None:
    text = path.read_text(encoding="utf-8")
    names = [line.strip() for line in text.splitlines()]
    print("Students:", len(names))
    Path("report.txt").write_text("\n".join(names), encoding="utf-8")
```

### Separated steps

```
from pathlib import Path

def read_names(path: Path) -> list[str]:
    text = path.read_text(encoding="utf-8")
    return [line.strip() for line in text.splitlines()]

def write_report(path: Path, names: list[str]) -> None:
    path.write_text("\n".join(names), encoding="utf-8")
```

> **Using the separate functions**
>
> After defining read_names() and write_report(), the main function can coordinate the steps.
>
> ```
> def main() -> None:
>     input_path = Path("students.txt")
>     output_path = Path("report.txt")
>     names = read_names(input_path)
>     write_report(output_path, names)
>     print(f"Saved {len(names)} names to {output_path}")
>
> if __name__ == "__main__":
>     main()
> ```
>
> This snippet assumes Path, read_names() and write_report() have already been imported or defined earlier in the file.

Split code when the steps can be understood or tested independently; not every two-line block needs its own function.

---

## 3. Reduce duplicated code

When the same logic is repeated, a fix must be applied in multiple places. A function can represent the shared logic.

### Repeated logic

```
price_a = 10 * 1.2
price_b = 15 * 1.2
price_c = 22 * 1.2
```

### Shared function

```
def with_tax(price):
    return price * 1.2

price_a = with_tax(10)
price_b = with_tax(15)
price_c = with_tax(22)
```

---

## 4. Handle errors meaningfully

Catch errors you can handle. Silently ignoring an exception can hide damaged data or a real problem.

### Avoid

```
try:
    points = int(user_input)
except:
    pass
```

### Specific error

```
try:
    points = int(user_input)
except ValueError:
    print("Enter a whole number.")
```

---

## 5. Comment on why, not the obvious line

A comment should add context or explain a decision that is not clear from the code itself, rather than restating the command.

### A comment that adds little

```
# Increase attempts by one
attempts += 1
```

### A comment that explains why

```
# Allow one retry after a temporary network failure.
attempts += 1
```

---

## 6. Separate independent parts of a program

As a project grows, group logic into modules by responsibility. User interface, file operations and calculations do not all need to live in one file.

```
project/
    main.py          # program entry point
    file_reader.py   # reading input files
    calculations.py  # calculation logic
    report.py        # formatting the output
```

This is one possible layout, not a requirement for every small script. Structure should match the project's size.

---

## 7. Use type hints where they improve clarity

Type annotations document expected inputs and outputs. They do not automatically validate values at runtime.

```
def calculate_average(values: list[float]) -> float:
    if not values:
        raise ValueError("values must not be empty")
    return sum(values) / len(values)
```

---

These are guidelines, not mechanical prohibitions. Apply them to the problem and keep a consistent style throughout the project.